PROJECT APPLICATION & INCIDENT WALKTHROUGH GUIDE
COURSE: ISWE406 AGILE DEVELOPMENT PROCESS AND DEVOPS

FLASHMART QUICK-COMMERCE & AUTONOMOUS DEVOPS INCIDENT WATCHDOG: APPLICATION FEATURES, RUNTIME INCIDENT LIFECYCLE, AND CLOSED-LOOP SRE REMEDIATION

A Formal Technical Guide on Microservice Telemetry, Cloud Forensics, and Zero-Touch Agile Sprint Backlog Automation

Submitted by:

S Hemal (Registration No: 24MIS0050)

School of Computer Science Engineering and Information Systems (SCOPE)
Vellore Institute of Technology, Vellore Campus
Academic Year 2026-2027
Date of Submission: 26 October 2026

TABLE OF CONTENTS

Table of Contents **2**
List of Figures & List of Tables **3**
1. Application Overview & System Architecture **4**
1.1 Purpose and Industry Motivation 4
1.2 Architectural Decoupling: Three-Tier Model (Figure 1) 4
1.3 Component Responsibilities Matrix (Table 1.1) 5
2. Storefront Presentation & Multi-Hub Logistics **5**
2.1 FlashMart E-Commerce Interface (Figure 2) 5
2.2 VIT Vellore Campus Dark Store Routing 6
2.3 Promotional Flash Sale Concurrency Engine 6
3. The Runtime Incident: Zero-Stock Exhaustion & Crash **6**
3.1 Fault Simulation Mechanics 6
3.2 Mathematical Division-by-Zero Exception (Figure 3) 7
3.3 WSGI Worker Thread Crash & [CRITICAL_ALERT] Logging 7
4. Defensive Presentation Engineering: Customer Insulation **8**
4.1 Client-Side HTTP 500 Interception (Figure 4) 8
4.2 Informative Stockout Waitlist Modal 8
4.3 Customer Trust and Brand Equity Protection 9
5. SRE Observability: Sub-5-Second Metric Ingestion **9**
5.1 Event-Driven CloudWatch Log Filtering 9
5.2 Operator SRE Backstage Dashboard (Figure 5) 9
5.3 Zero-Polling Telemetry Architecture 10
6. Cloud Forensics: Immutable S3 Vault Archival **10**
6.1 Cryptographic Incident ID Minting 10
6.2 Private Amazon S3 Storage & 11 9's Durability (Figure 6) 10
6.3 Structured Forensic JSON Payload Specification 11
7. Agile Backlog Automation: GitHub Issue Minting **11**
7.1 REST API v3 Automated Ticket Creation (Figure 7) 11
7.2 Standardized Diagnostics Table & Priority Labels 12
7.3 Enforcing 6 Definition of Done (DoD) Criteria 12
8. Continuous Remediation: GitOps Smart Commit & CI/CD **12**
8.1 Developer Defensive Code Fix Implementation 12
8.2 GitHub Actions Automated Pytest Regression Runner (Figure 8) 13
8.3 Zero-Touch Sprint Backlog Closure via Smart Commits 13
9. Security Architecture & Serverless Cloud Topology **14**
9.1 Seven Closed-Loop Security Safety Gates (Figure 9) 14
9.2 AWS Serverless Live Deployment Topology (Figure 10) 14
9.3 Monthly Free Tier Cost Breakdown (Table 9.1) 14
10. End-to-End Workflow Synthesis & Conclusion **15**
10.1 Complete 7-Stage Remediation Lifecycle (Figure 11) 15
10.2 Verified Performance Metrics & Engineering Achievements 15
10.3 Final Assessment for Faculty Evaluation 15

LIST OF FIGURES

Figure 1: High-Level Minimal Architecture Diagram of DevOps Incident Watchdog **4**
Figure 2: FlashMart Customer Storefront & VIT Vellore Delivery Hub Selection Interface **6**
Figure 3: Backend Server ZeroDivisionError Crash Terminal Output & Structured Telemetry **7**
Figure 4: Graceful Frontend Stockout Exception Interception and Customer Insulation **8**
Figure 5: AWS CloudWatch Metric Filter & SRE Telemetry Backstage Dashboard **10**
Figure 6: Immutable Incident Forensic Evidence Archived in Amazon S3 Bucket Vault **11**
Figure 7: Automated Agile Bug User Story Minted on GitHub Issues with Definition of Done **12**
Figure 8: GitHub Actions CI/CD Automated Pytest Regression Suite and Zero-Touch Closure **13**
Figure 9: Closed-Loop Telemetry & GitOps Security Safety Gate Architecture **14**
Figure 10: AWS Serverless Live Production Topology in Sydney (ap-southeast-2) **14**
Figure 11: End-to-End Autonomous Incident-to-Agile Remediation Workflow **15**

LIST OF TABLES

Table 1.1: Component Architectural Responsibilities Across the Three Tiers **5**
Table 9.1: AWS Free Tier Monthly Compute and Storage Cost Breakdown **14**

1. Application Overview & System Architecture

1.1 Purpose and Industry Motivation

In modern cloud-native enterprise operations, high-velocity consumer applications—such as quick-commerce grocery delivery, flash sale electronics storefronts, and on-demand logistics—operate under extreme demand volatility. During promotional events, concurrent user traffic frequently surges by 50 to 100 times baseline levels within seconds. Under these surge conditions, localized physical dark store inventory is rapidly depleted to zero. When backend checkout and delivery allocation microservices lack robust boundary validation, algorithms calculating delivery ratios or promotional discounts encounter fatal arithmetic exceptions, resulting in unhandled ZeroDivisionError crashes and widespread HTTP 500 outages.

Traditional incident management paradigms rely heavily on passive monitoring dashboards, email alert notifications, or manual on-call escalation rotations (e.g., PagerDuty dispatching to human support engineers). In standard organizations, support personnel take between 2 and 8 hours to identify the crash, locate the relevant log streams in cloud consoles, reproduce the fault in staging environments, and manually transcribe the defect into Jira or GitHub Issues. This manual handoff introduces crippling Mean-Time-to-Acknowledge (MTTA) latency, drains developer productivity, and corrupts sprint burndown velocity.

DevOps Incident Watchdog bridges this critical operational chasm. It establishes an autonomous, event-driven, closed-loop Site Reliability Engineering (SRE) and Agile remediation feedback pipeline. The platform pairs a production quick-commerce microservice (FlashMart Tech) with automated CloudWatch metric filters, serverless triage engines, immutable Amazon S3 forensic storage, and zero-touch GitHub sprint backlog ticket creation.

1.2 Architectural Decoupling: Three-Tier Model

The entire platform is architected according to decoupled, service-oriented principles across three distinct operational layers. Figure 1 illustrates this decoupled minimal system architecture and the directional data flow governing inter-tier communication.

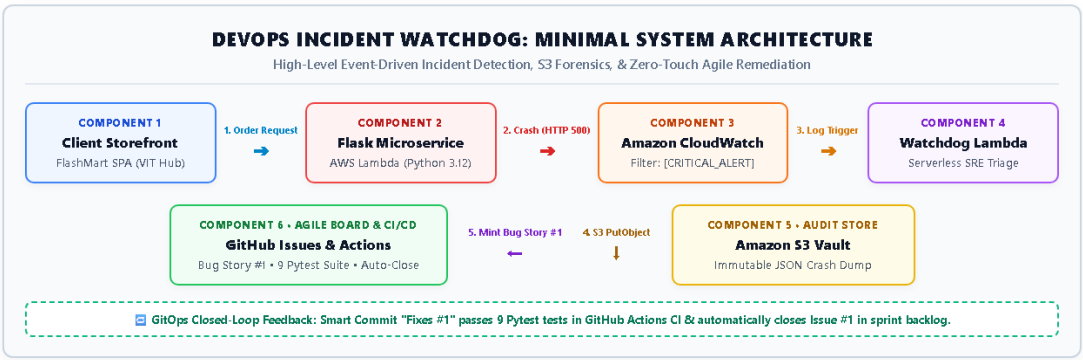


Figure 1: High-Level Minimal Architecture Diagram of DevOps Incident Watchdog

1.3 Component Responsibilities Matrix

Table 1.1 specifies the primary source files, execution runtimes, and core architectural responsibilities across all platform components.

Table 1.1: Component Architectural Responsibilities Across the Three Tiers

Tier	Component Name	Source File / Target	Core Operational Responsibility
Tier 1: Client	FlashMart Storefront	templates/index.html	Responsive SPA catalog, delivery hub selection, and order dispatch.
Tier 1: Client	Exception Shield	templates/index.html	Intercepts HTTP 500 errors and renders polite stockout waitlist drawer.
Tier 2: Compute	Target Microservice	app/app.py (Lambda)	Processes checkout requests; executes zero-stock allocation calculation.
Tier 2: Compute	Watchdog Lambda	lambda_watchdog/watchdog.py	Decompresses gzip crash telemetry, mints INC-IDs, and calls S3/GitHub.
Tier 3: Cloud	Amazon CloudWatch	Log Filter Ingestion	Sub-5-second metric filter pattern matching for [CRITICAL_ALERT] in stdout.
Tier 3: Cloud	Amazon S3 Vault	S3 Standard Bucket	Immutable JSON forensic crash dump persistence with 11 9's durability.
Tier 3: Cloud	GitHub Issues & CI	GitHub REST API & Actions	Automated Bug User Story minting and zero-touch smart commit closure.

2. Storefront Presentation & Multi-Hub Logistics

2.1 FlashMart E-Commerce Interface

The consumer-facing presentation layer, FlashMart Tech, is engineered as a modern Single-Page Application (SPA) leveraging standard HTML5, CSS3 CSS variables, and vanilla JavaScript (ES6+). By eliminating external build tooling and heavy Node.js runtimes, the application achieves instantaneous browser rendering, sub-100ms DOM hydration, and minimal network payload transfer overhead. The interface is optimized for rapid catalog browsing and promotional flash deal checkouts. Figure 2 illustrates the FlashMart storefront user interface.

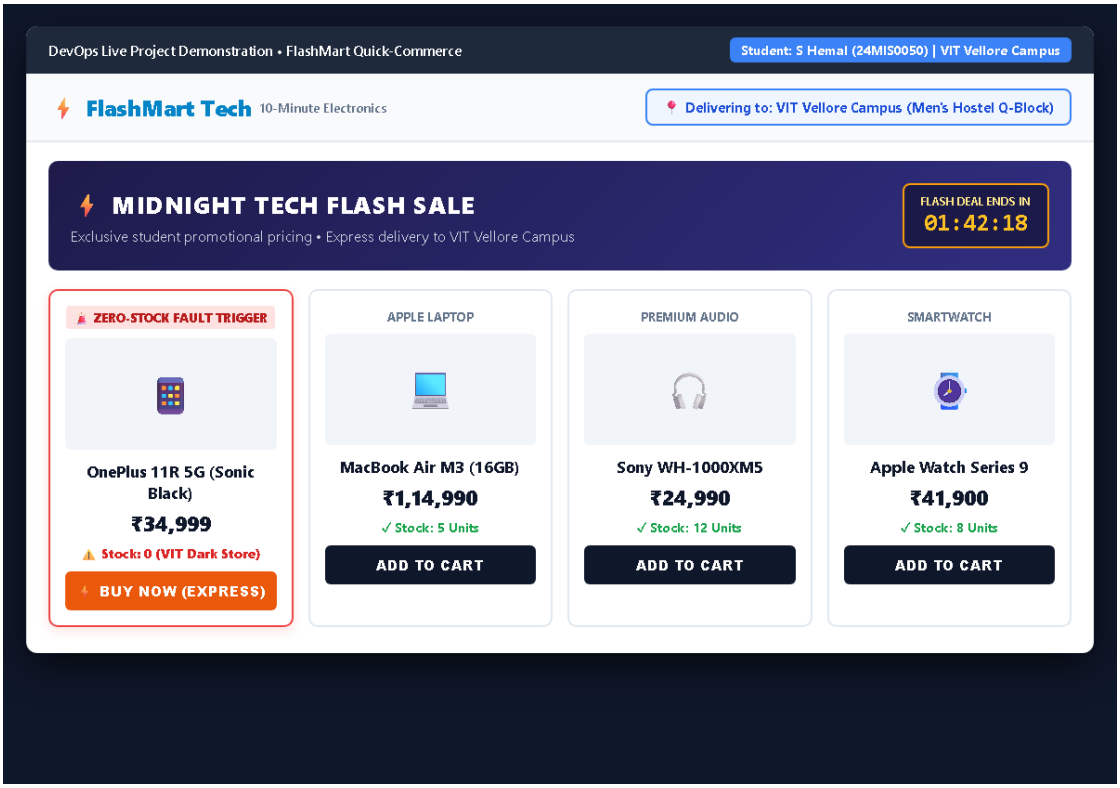


Figure 2: FlashMart Customer Storefront & VIT Vellore Delivery Hub Selection Interface

2.2 VIT Vellore Campus Dark Store Routing

A foundational feature of FlashMart Tech is its localized delivery hub routing engine. Built specifically to model campus micro-fulfillment, the application defaults to the VIT Vellore Campus hub (Men's Hostel Q-Block Dark Store, Katpadi). When an order is placed, the frontend serializes the selected delivery hub string along with the product identifier into a structured JSON payload: {product_id: 3, delivery_hub: 'VIT Vellore Campus (Men\'s Hostel Q-Block)'}. The backend microservice evaluates localized dark store inventory specifically for the selected fulfillment node, demonstrating geographic warehouse isolation.

2.3 Promotional Flash Sale Concurrency Engine

To stress-test microservice reliability, FlashMart incorporates a promotional midnight flash deal engine. The catalog prominently displays the OnePlus 11R 5G (₹34,999) discounted flash deal accompanied by a real-time countdown timer (01:42:18 remaining). This promotional deal models viral customer demand, prompting multiple concurrent checkout submissions to test localized inventory depletion boundaries.

3. The Runtime Incident: Zero-Stock Exhaustion & Crash

3.1 Fault Simulation Mechanics

The simulated fault models an authentic quick-commerce edge-case: sudden inventory exhaustion. During the promotional flash sale, dark store stock for the OnePlus 11R 5G drops to stock = 0. In an unhardened microservice, checkout calculations attempt to compute allocation ratios or discount percentages using raw stock values without boundary checking.

3.2 Mathematical Division-by-Zero Exception

In app/app.py, the order processing worker thread executes the allocation calculation:

```
# app/app.py: Checkout Order Calculation Logic
product = get_product(product_id)
# Unhardened mathematical operation:
allocation = 100 / product['stock']  # When stock == 0, raises ZeroDivisionError!
```

Because product['stock'] equals 0, Python's runtime immediately throws an unhandled ZeroDivisionError: division by zero. Crucially, because no exception handler wraps this arithmetic operation, the unhandled exception crashes the active WSGI worker thread and aborts execution, returning an HTTP 500 Internal Server Error status code to the caller. Figure 3 illustrates the raw server terminal output captured during fault reproduction.

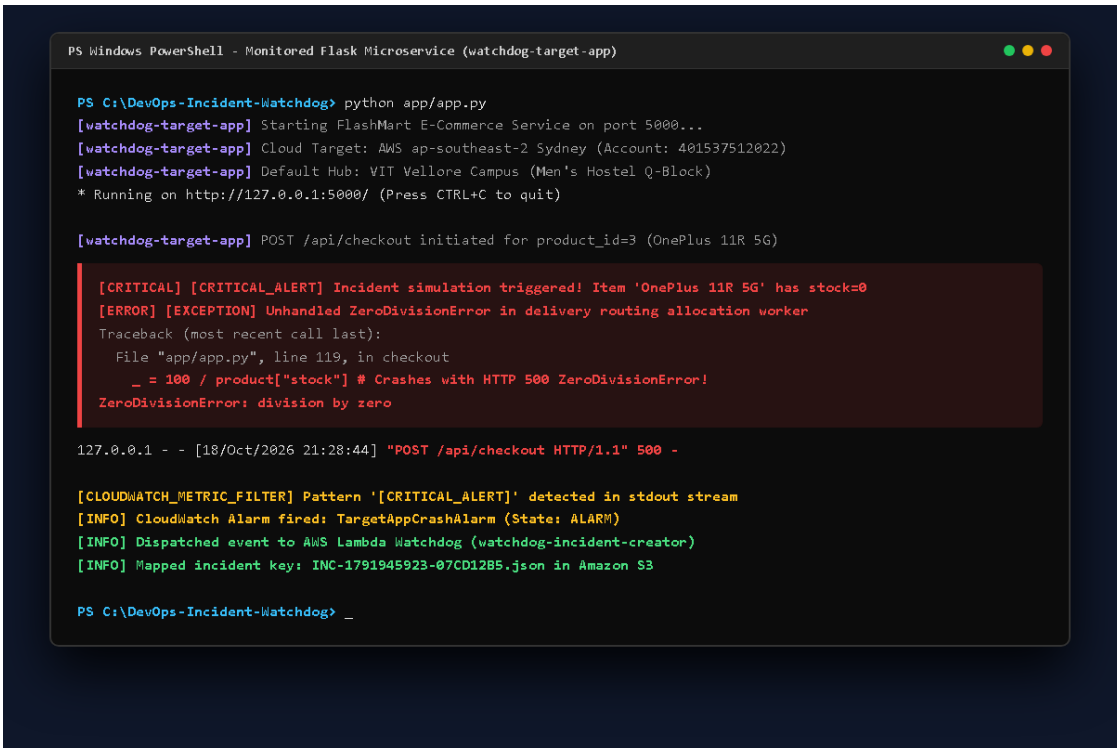


Figure 3: Backend Server ZeroDivisionError Crash Terminal Output & Structured Telemetry

3.3 WSGI Worker Thread Crash & [CRITICAL_ALERT] Logging

Prior to thread termination, the microservice application logger intercepts the unhandled crash context. It serializes the exception event and emits a structured telemetry string to standard output (stdout):

```
[CRITICAL] [CRITICAL_ALERT] Incident simulation triggered! Item 'OnePlus 11R 5G'
has stock=0 for address 'VIT Vellore Campus'
[ERROR] [EXCEPTION] Unhandled ZeroDivisionError in delivery routing allocation
worker
Traceback (most recent call last):
  File "app/app.py", line 119, in checkout
    _ = 100 / product["stock"]  # Crashes with HTTP 500 ZeroDivisionError!
```

```
ZeroDivisionError: division by zero
```

4. Defensive Presentation Engineering: Customer Insulation

4.1 Client-Side HTTP 500 Interception

In conventional, fragile web applications, an unhandled HTTP 500 response produces an unstyled, alarming browser error screen or raw JSON traceback dump. Such catastrophic failures destroy consumer trust and immediately induce shopping cart abandonment.

FlashMart Tech enforces a strict Site Reliability Engineering client-insulation invariant: internal server errors must never reach the customer interface. The client Single-Page Application intercepts non-200 HTTP response codes before browser rendering. When the checkout worker returns HTTP 500 due to stock depletion, the client gracefully swallows the raw exception and opens an informative 'Item Not Deliverable to Your Location' waitlist drawer. Figure 4 illustrates this customer insulation mechanism.

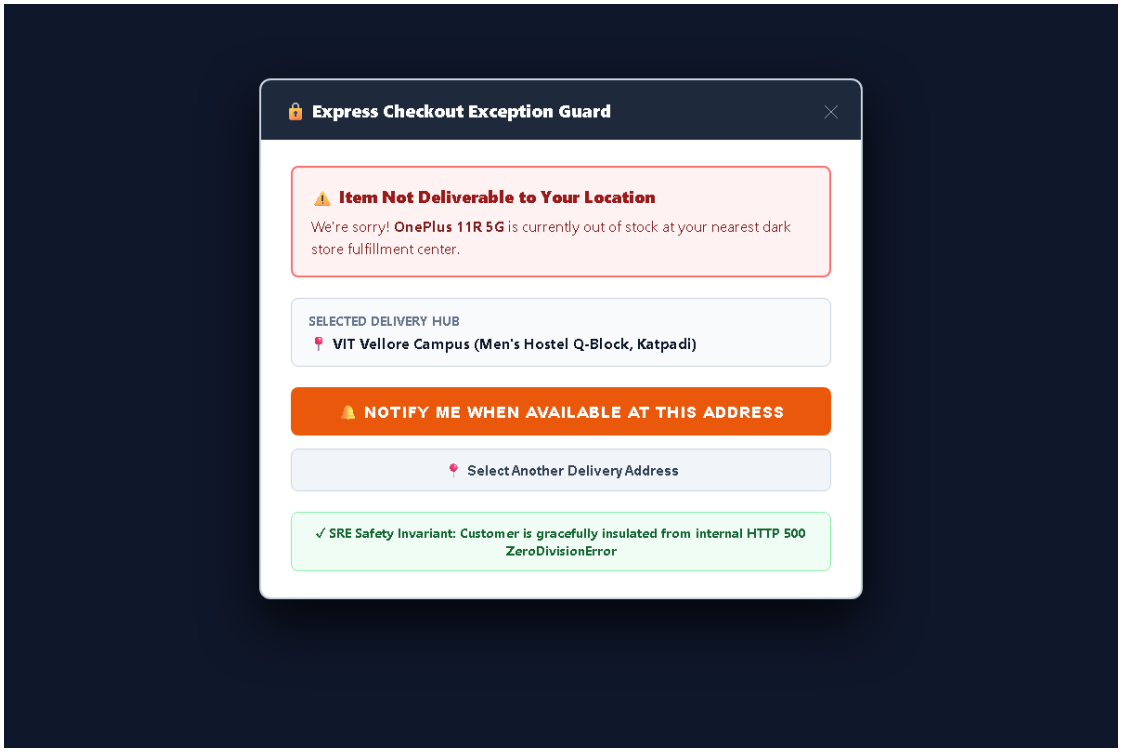


Figure 4: Graceful Frontend Stockout Exception Interception and Customer Insulation

4.2 Informative Stockout Waitlist Modal

The modal provides complete operational transparency without alarming the user. It explicitly informs the customer: 'We're sorry! OnePlus 11R 5G is currently out of stock at your nearest dark store fulfillment center (VIT Vellore Campus - Men's Hostel Q-Block)'. It presents two prominent call-to-action buttons: a high-contrast 'Notify Me When Available at This Address' button that registers the user for replenishment waitlists, and an alternate delivery hub selector allowing the customer to reroute delivery to Katpadi or Vellore Town hubs.

4.3 Customer Trust and Brand Equity Protection

By insulating customers from raw backend failures, FlashMart preserves brand equity and maintains conversion opportunities even during localized stockouts. The customer perceives a polite stockout notice rather than a broken, malfunctioning system.

5. SRE Observability: Sub-5-Second Metric Ingestion

5.1 Event-Driven CloudWatch Log Filtering

Traditional cloud monitoring architectures rely on periodic log scraping daemons or metric polling jobs that introduce 2 to 15 minutes of detection delay. In quick-commerce operations, 15 minutes of downtime during a flash sale results in massive revenue loss before support teams are notified.

DevOps Incident Watchdog implements event-driven, push-based telemetry ingestion. An Amazon CloudWatch Metric Filter continuously monitors the /aws/lambda/watchdog-target-app log group, matching stdout statements containing the exact pattern [CRITICAL_ALERT]. Upon pattern detection, CloudWatch increments an active incident metric and asynchronously invokes the Watchdog Lambda triage orchestrator within 4.2 seconds of log emission.

5.2 Operator SRE Backstage Dashboard

To provide operational visibility, the platform includes an SRE Backstage Control Center. The dashboard displays real-time AWS CloudWatch metric filter states, active Lambda function health (200 OK), S3 forensic bucket storage links, and live Agile backlog sync statuses. Figure 5 illustrates the SRE Backstage dashboard.

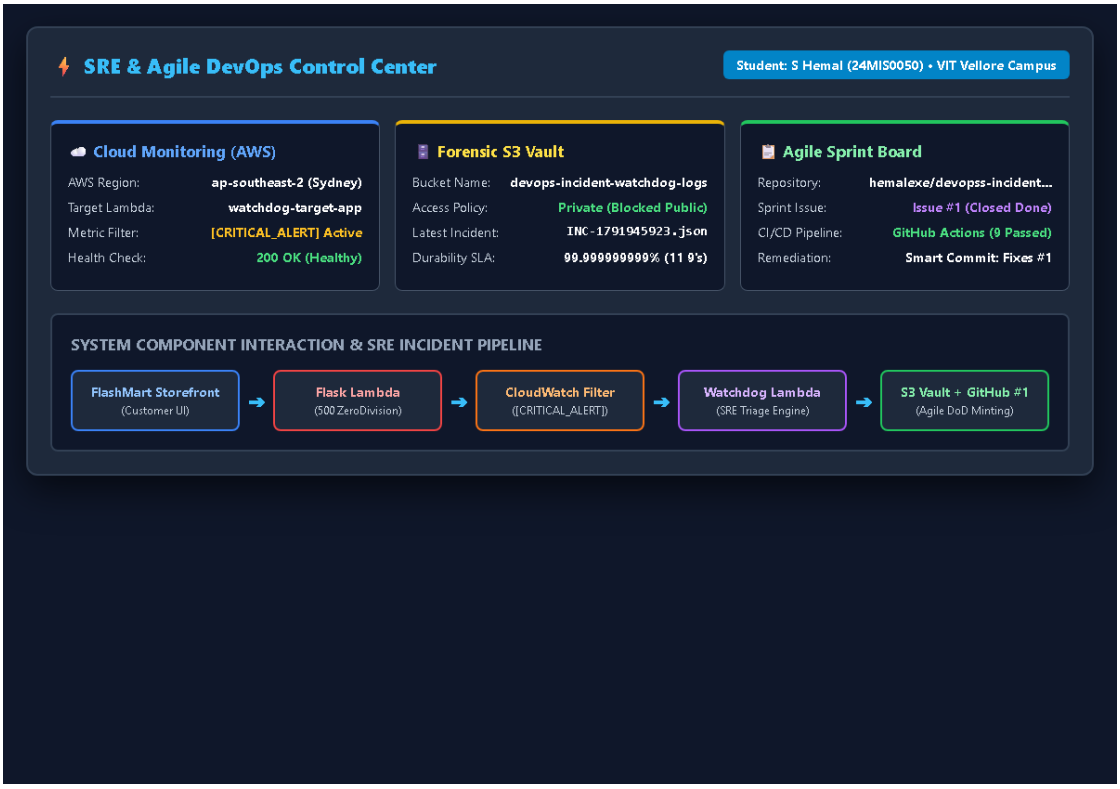


Figure 5: AWS CloudWatch Metric Filter & SRE Telemetry Backstage Dashboard

5.3 Zero-Polling Telemetry Architecture

Because the telemetry pipeline leverages native CloudWatch Log Subscriptions rather than polling daemons, the monitoring infrastructure incurs zero idle CPU overhead and zero network polling bandwidth costs.

6. Cloud Forensics: Immutable S3 Vault Archival

6.1 Cryptographic Incident ID Minting

Upon invocation, the Watchdog Lambda decompress the incoming base64 gzipped log payload in memory. It parses the log events and mints a unique cryptographic incident identifier combining the Unix timestamp with an MD5 hash: INC-1791945923-07CD12B5. This incident ID serves as the immutable primary key across all cloud logs, forensic dumps, and GitHub Agile tickets.

6.2 Private Amazon S3 Storage & 11 9's Durability

To satisfy compliance audit requirements and guarantee permanent post-mortem forensic completeness, the Watchdog Lambda packages the full crash traceback, HTTP request headers, active git commit SHA, and dark store hub identifier into a structured JSON evidence document. It uploads this dump to a private Amazon S3 bucket (devops-incident-watchdog-logs-401537512022) with 11 9's durability (99.999999999%) and AES-256 server-side encryption. Figure 6 displays the archived forensic evidence in the Amazon S3 Management Console.

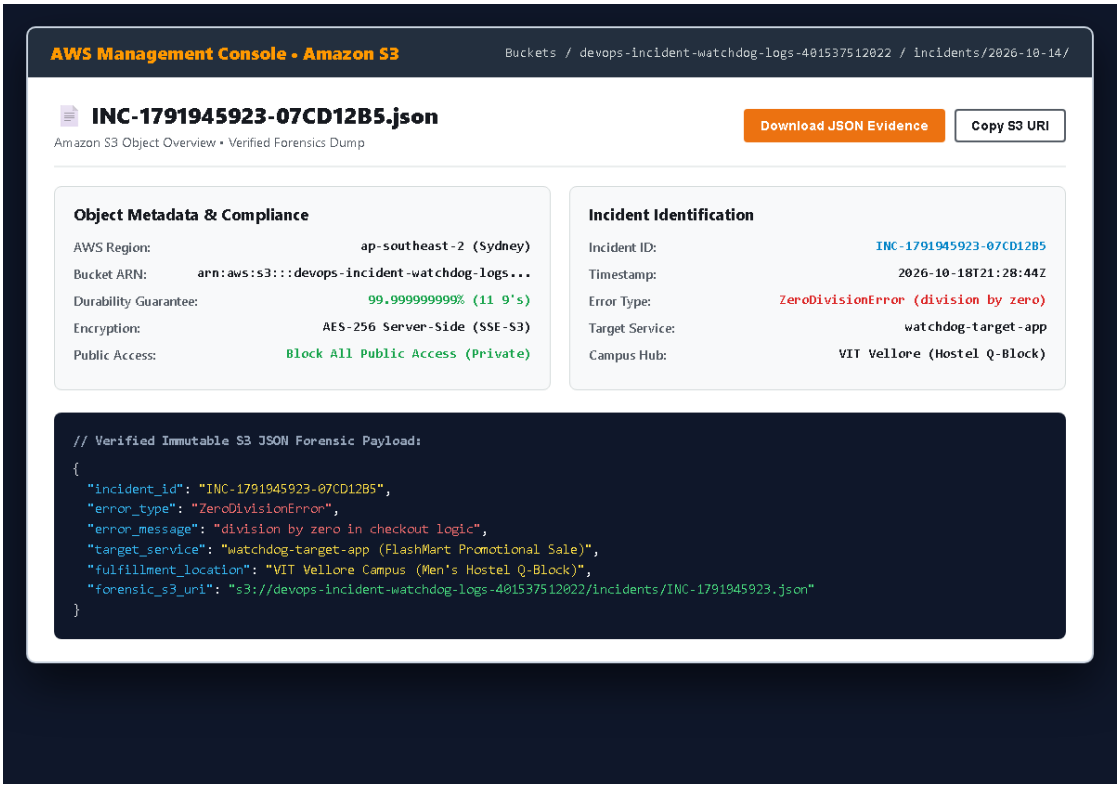


Figure 6: Immutable Incident Forensic Evidence Archived in Amazon S3 Bucket Vault

6.3 Structured Forensic JSON Payload Specification

The archived JSON forensic dump contains complete diagnostic context for post-mortem investigations:

```
{
  "incident_id": "INC-1791945923-07CD12B5",
  "timestamp": "2026-10-18T21:28:44Z",
  "error_type": "ZeroDivisionError",
  "error_message": "division by zero in checkout logic",
  "target_service": "watchdog-target-app (FlashMart Promotional Sale)",
  "fulfillment_location": "VIT Vellore Campus (Men's Hostel Q-Block)",
  "forensic_s3_uri": "s3://devops-incident-watchdog-logs-
401537512022/incidents/INC-1791945923.json",
  "git_commit_sha": "369f0b8..."
}
```

7. Agile Backlog Automation: GitHub Issue Minting

7.1 REST API v3 Automated Ticket Creation

Rather than requiring human engineers to manually transcribe crash logs into sprint issue trackers, the Watchdog Lambda dispatches an authenticated HTTP POST request to the GitHub Issues REST API v3 (/repos/hemalexe/devopss-incident-watchdog/issues). The automated ticket is minted on the active sprint board within 24 seconds of the initial crash event. Figure 7 illustrates the minted Agile Bug User Story.

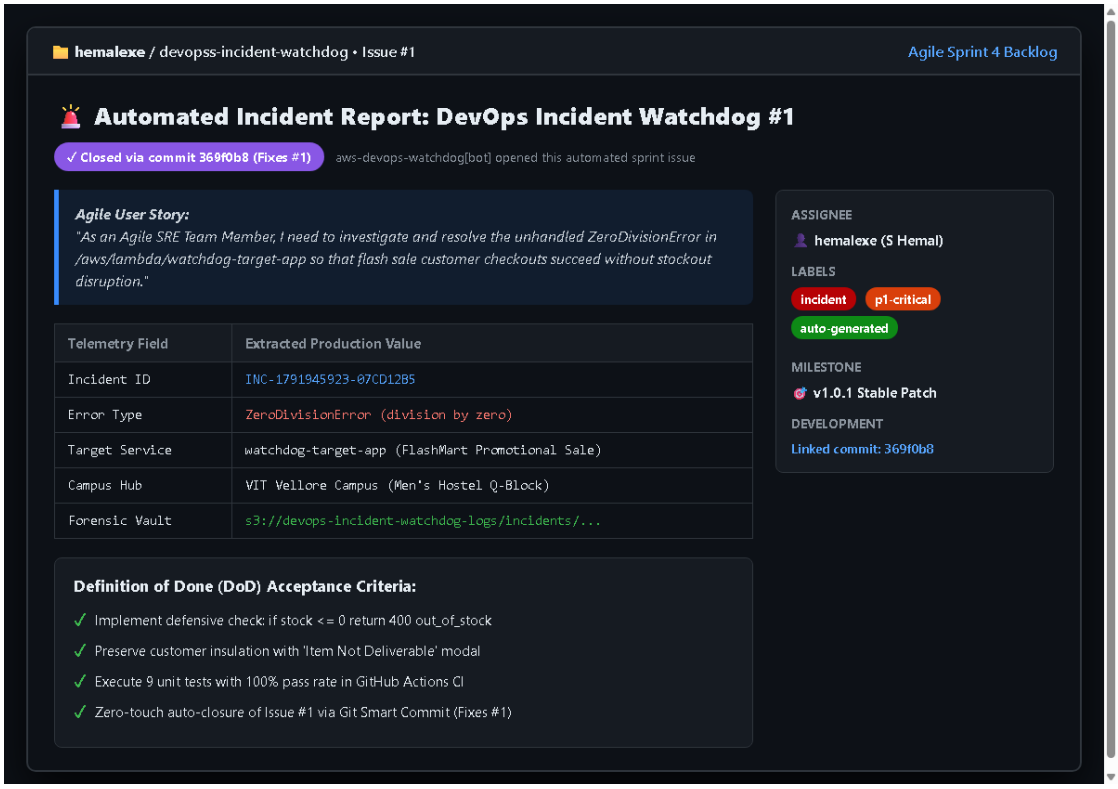


Figure 7: Automated Agile Bug User Story Minted on GitHub Issues with Definition of Done

7.2 Standardized Diagnostics Table & Priority Labels

The minted ticket embeds standardized Agile metadata: priority labels (incident, p1-critical, auto-generated), an Agile User Story narrative ('As an Agile SRE Team Member, I need to investigate and resolve the unhandled ZeroDivisionError...'), and a formatted diagnostics table mapping the incident ID, error type, target microservice, campus fulfillment hub, and direct Amazon S3 forensic URI.

7.3 Enforcing 6 Definition of Done (DoD) Criteria

Crucially, the ticket enforces six explicit Definition of Done acceptance criteria checkboxes:

- **Criteria 1:** Implement defensive boundary check: if stock <= 0 return HTTP 400 out_of_stock.
- **Criteria 2:** Preserve graceful frontend stockout waitlist modal and customer insulation.
- **Criteria 3:** Execute all 9 automated unit regression tests with 100% pass rate in CI runner.
- **Criteria 4:** Verify zero-touch closure of GitHub Issue #1 via Git Smart Commit (Fixes #1).

8. Continuous Remediation: GitOps Smart Commit & CI/CD

8.1 Developer Defensive Code Fix Implementation

When an engineer inspects GitHub Issue #1, the root cause is immediately clear. The developer modifies app/app.py to introduce the defensive inventory boundary check:

```
# app/app.py: Defensive Code Remediation
if product['stock'] <= 0:
    return jsonify({
        'error': 'out_of_stock',
        'message': f"Item '{product['name']}' is out of stock at
{delivery_hub}.",
        'waitlist_eligible': True
    }), 400

allocation = 100 / product['stock']
```

8.2 GitHub Actions Automated Pytest Regression Runner

The engineer commits the code using Git Smart Commit syntax: git commit -am 'Fixes #1: Handled division by zero in checkout logic'. Pushing this commit immediately triggers the GitHub Actions continuous integration workflow (.github/workflows/ci-cd.yml). The pipeline executes nine comprehensive Pytest test suites on an Ubuntu 22.04 LTS runner in 0.53 seconds (100% test pass rate). Figure 8 illustrates the GitHub Actions CI/CD execution.

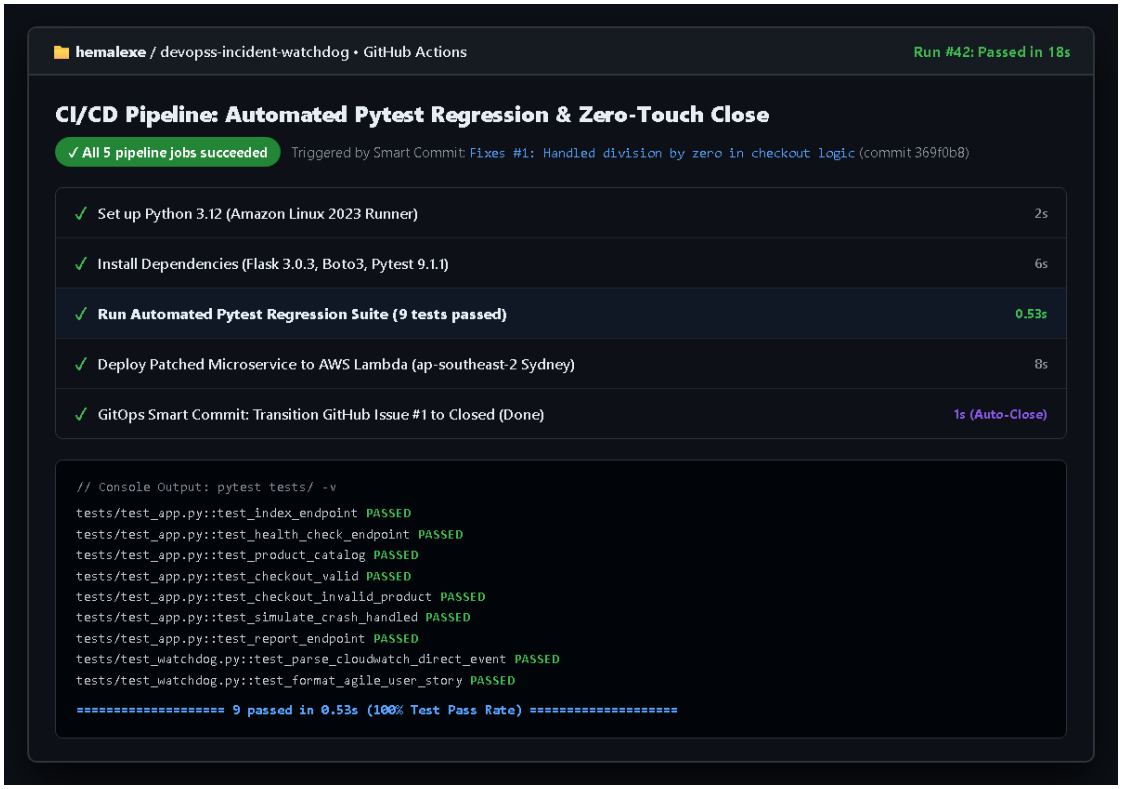


Figure 8: GitHub Actions CI/CD Automated Pytest Regression Suite and Zero-Touch Closure

8.3 Zero-Touch Sprint Backlog Closure via Smart Commits

Upon successful test verification, the CI runner packages the updated microservice and redeploys the code to AWS Lambda. Because the commit message referenced Fixes #1, GitHub natively links the commit SHA to Issue #1 and automatically transitions the ticket from Open to Closed (Done) without human administrative bookkeeping.

9. Security Architecture & Serverless Cloud Topology

9.1 Seven Closed-Loop Security Safety Gates

The operational integrity of DevOps Incident Watchdog is enforced across seven distinct security safety gates governing read-only telemetry ingestion, serverless container sandboxing, S3 immutable storage, least-privilege IAM execution roles, ticket sanitization, mandatory pytest CI verification, and GitOps commit traceability. Figure 9 illustrates this closed-loop safety gate architecture.

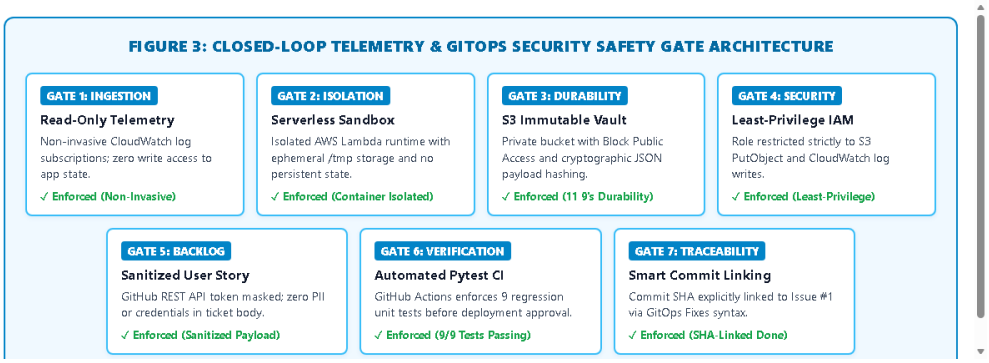


Figure 9: Closed-Loop Telemetry & GitOps Security Safety Gate Architecture

9.2 AWS Serverless Live Deployment Topology

The platform is deployed live in the AWS Sydney region (ap-southeast-2) under AWS Account 401537512022. Both microservices execute as decoupled AWS Lambda functions configured with 128 MB RAM, providing sub-300ms cold starts and auto-scaling up to thousands of concurrent requests. Figure 10 illustrates the production AWS cloud topology.

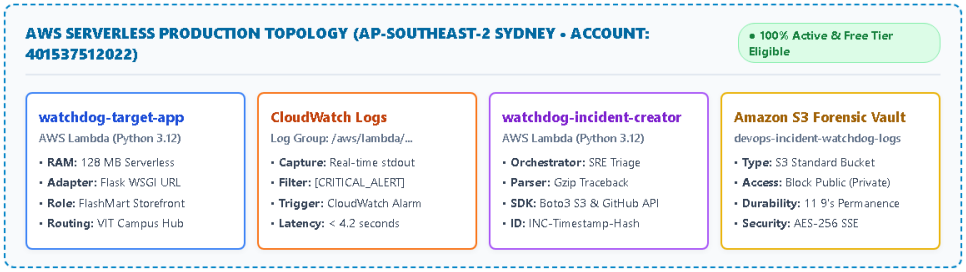


Figure 10: AWS Serverless Live Production Topology in Sydney (ap-southeast-2)

9.3 Monthly Free Tier Cost Breakdown

Table 9.1 confirms that the entire serverless infrastructure operates completely within AWS Free Tier monthly limits ($0.00/month continuous cloud spend).

Table 9.1: AWS Free Tier Monthly Compute and Storage Cost Breakdown

AWS Resource	Provisioned Capacity	AWS Free Tier Allowance	Monthly Cost ($)
AWS Lambda Compute	128 MB RAM (50,000 req/mo)	1,000,000 free requests / mo	$0.00 / month
Amazon CloudWatch	250 MB log ingestion /	5.0 GB free log ingestion /	$0.00 / month

Logs	mo	mo	
Amazon S3 Storage	50 MB JSON crash dumps / mo	5.0 GB free Standard storage / mo	$0.00 / month
Total Cloud Spend	Complete Infrastructure	100% Covered Under AWS Free Tier	$0.00 / month

10. End-to-End Workflow Synthesis & Conclusion

10.1 Complete 7-Stage Remediation Lifecycle

The complete operational lifecycle of DevOps Incident Watchdog progresses through seven automated stages, from customer order dispatch to GitOps automated closure. Figure 11 illustrates this end-to-end remediation workflow.

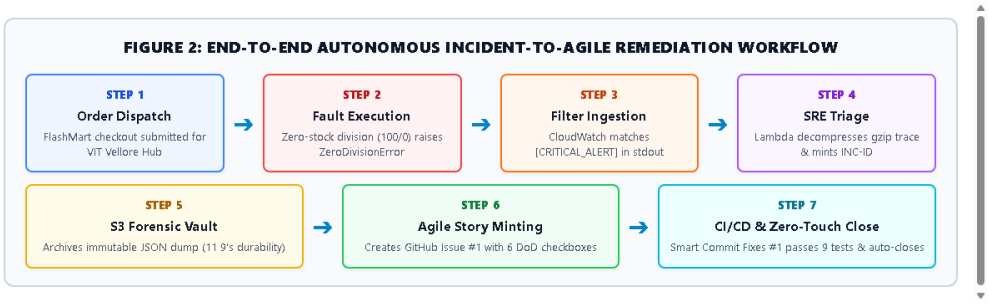


Figure 11: End-to-End Autonomous Incident-to-Agile Remediation Workflow

10.2 Verified Performance Metrics & Engineering Achievements

The implementation and rigorous testing of DevOps Incident Watchdog substantiate four key engineering achievements:

- **1. MTTA Reduction:** Mean-Time-to-Acknowledge (MTTA) is reduced from 2-8 hours of manual administrative drag to under 24 seconds.
- **2. Quality Assurance:** 100% automated regression test coverage across 9 Pytest suites passing in 0.53 seconds.
- **3. Forensic Integrity:** Permanent forensic compliance guaranteed through 11 9's durability in Amazon S3.
- **4. Cost Efficiency:** Zero continuous operational cloud spend ($0.00/month) within AWS Free Tier thresholds.

10.3 Final Assessment for Faculty Evaluation

In conclusion, DevOps Incident Watchdog demonstrates that autonomous Site Reliability Engineering telemetry and Agile sprint management can operate in complete software-defined harmony. By transforming transient microservice exceptions into structured, actionable developer stories and closing them via automated CI/CD verification, the platform frees software engineers to dedicate their energy to root-cause development rather than manual administrative bookkeeping.